

TierMoE: Batch-Aware Expert Placement for Memory-Tiered MoE Inference

Vinay Jumani

Department of Computer Science and Information Systems, BITS Pilani, K. K. Birla Goa Campus, Goa 403726, India

Email: f20210848@goa.bits-pilani.ac.in | Date: September 2026

Abstract—Mixture-of-Experts (MoE) architectures achieve frontier-class reasoning capacity by decoupling model parameter scale from per-token compute FLOPs via sparse dynamic routing. However, serving large MoE foundation models in high-throughput production environments quickly exhausts fast GPU High-Bandwidth Memory (HBM). Emerging Compute Express Link (CXL) Type-3 memory expanders offer a cost-effective, multi-terabyte secondary memory tier, but their orders-of-magnitude lower link bandwidth (16–64 GB/s) creates severe memory-stall bottlenecks whenever uncoordinated expert migrations occur. Existing expert offloading techniques rely on static global popularity pinning, single-request isolated optimization, sequence-level temporal lookahead, or reactive hardware LRU caching. When multiple inference requests execute concurrently, their conflicting expert requirements induce working set explosion and catastrophic intra-batch cache thrashing. In this paper, we present *TierMoE*, a batch-aware expert placement framework specifically architected for memory-tiered MoE serving. Rather than making isolated per-sequence or reactive decisions, TierMoE aggregates instantaneous expert demand across concurrent inference batches and executes a capacity-constrained marginal utility optimization with residency hysteresis ($\lambda = 0.5$). We evaluate TierMoE across both controlled synthetic workloads and 461,184 authentic physical routing decisions profiled from *Qwen3-30B-A3B* (128 experts, top-8 routing, 48 layers) on an NVIDIA RTX A6000 testbed across conversational dialogue (ShareGPT) and mathematical reasoning (GSM8K). In capacity-constrained regimes, TierMoE achieves a +5.02% to +8.68% higher HBM hit rate and reduces CXL parameter traffic by 13.20% to 20.44% compared to single-request controls ($p < 0.001$). Against published baseline paradigms, TierMoE outperforms predictive sequence lookahead by +6.99% ($p < 0.001$) and reactive CXL-LRU tiering by +16.61% to +32.27% by completely eliminating intra-batch thrashing. Crucially, empirical testing on authentic traces *falsifies* the hypothesis that pairwise expert co-activation tracking outperforms pure batch frequency (-1.25% , $p = 0.259$), proving that instantaneous batch frequency avoids historical matrix inertia while solving in just 51.2 μs per step. Finally, CXL interconnect sensitivity analysis across 16–64 GB/s demonstrates that link bandwidth is the primary physical bottleneck (4.00 $\times$ transfer time scaling), while discrete-event queue characterization using upstream *CXLMemSim* establishes that concurrent shared-link transfers incur queue startup overhead strictly once ($1.0 \times T_0 \pm 1.35\%$).

Keywords—Mixture-of-Experts (MoE), Compute Express Link (CXL), Memory Tiering, Multi-Tenant LLM Serving, Batch-Aware Scheduling, Cache Optimization.

1. Introduction

Mixture-of-Experts (MoE) architectures have rapidly emerged as the foundational architectural design for modern frontier large language models, powering state-of-the-art systems including Mixtral [1], DeepSeek-V3 [2], and Qwen2.5/Qwen3 [3]. The core architectural innovation of MoE models lies in replacing conventional dense feed-forward network (FFN) layers with an ensemble of specialized sub-networks ('experts') coordinated by a learned parametric gating router. For each input token, the router computes gating coefficients and dynamically activates only a sparse top- k subset of experts (e.g., $k = 2$ to 8 out of $N = 64$ to 256). This dynamic routing decouples total model parameter capacity from per-token floating-point operations (FLOPs), enabling unprecedented reasoning capability without proportional computational compute scaling during forward passes.

The Memory Capacity Chasm: While sparse compute reduces arithmetic execution costs, it does not reduce the *memory capacity footprint*. Every parameter of every expert must remain accessible during serving. In frontier architectures, the aggregate footprint of expert parameters reaches hundreds of gigabytes to multiple terabytes. For example, in the 128-expert *Qwen3-30B-A3B* architecture evaluated in this work, the 48 MoE layers encompass over **1.57 TiB** of expert weights in standard 16-bit precision. Accelerator High-Bandwidth Memory (HBM) remains severely capacity-constrained (typically 24 to 80 GB per GPU) and economically expensive. Storing entire frontier MoEs exclusively in GPU HBM requires multi-node expert parallelism, resulting in prohibitive hardware expenditures and low GPU compute utilization in low-to-medium throughput serving regimes.

Compute Express Link (CXL) as a Tiering Solution: Compute Express Link (CXL 2.0/3.0) Type-3 memory expanders offer a compelling architectural solution by providing byte-addressable, cache-coherent DDR5 memory pools accessible over PCIe 5.0/6.0 physical interfaces. CXL establishes a two-tier memory hierarchy: a fast, low-latency, capacity-constrained tier (GPU HBM) and a vast, cost-effective secondary capacity tier (CXL memory).

The Multi-Tenant Serving Bottleneck: While memory tiering resolves capacity limitations, the fundamental system bottleneck shifts to *interconnect transfer bandwidth*. A standard PCIe 5.0 $\times 16$ CXL link provides an effective bandwidth of 32 GB/s—approximately 30 to 60 $\times$ lower than GPU HBM bandwidth (1–2 TB/s). Transferring an uncompressed 256 MiB expert block over CXL requires approximately 8.39 ms, introducing severe execution stalls if required experts are not resident in fast GPU memory.

Existing expert management proposals generally fall into four paradigms: (1) *Static LFU popularity pinning* [4], (2) *Single-sequence greedy allocation*, (3) *Sequence-level predictive lookahead* (e.g., MoE-Infinity [6], ProMoE [7]), and (4) *Reactive hardware-managed caching / CXL-LRU tiering* (e.g., CXL-MoE [8]). While viable for isolated single-user offline inference, these approaches break down under **concurrent multi-tenant serving**. When an engine serves a batch of B concurrent requests (e.g., $B \in [8, 32]$), each request routes tokens to different expert subsets. This induces: (a) *working set explosion*, where aggregate demand rapidly exceeds fast memory; (b) *catastrophic intra-batch cache thrashing*, where requests within the exact same batch step evict each other's required experts; and (c) *predictive collisions*, where independent lookahead prefetch streams clobber shared HBM allocations.

Contributions: To resolve these challenges, this paper presents **TierMoE**, a batch-aware expert placement system that coordinates memory tiering across concurrent inference requests. The primary contributions are:

- **Batch-Aware Marginal Utility Solver:** We formulate the multi-tenant expert placement optimization problem and design *TierMoE-Batch-Aware-Greedy*, an ultra-lightweight solver combining multi-token batch frequency with residency hysteresis ($\lambda = 0.5$), executing in just **51.2 μ s per step** ($< 0.08\%$ of token latency).
- **Authentic Physical Model Profiling:** We instrument forward router hooks on dual NVIDIA RTX A6000 GPUs running *Qwen3-30B-A3B-Instruct-2507*, capturing **461,184 authentic physical routing decisions** across conversational dialogue (ShareGPT) and mathematical reasoning (GSM8K).
- **Rigorous Baseline Comparative Evaluation:** Across 24 capacity-constrained conditions on authentic traces, TierMoE outperforms predictive sequence lookahead by **+6.99%** hit rate ($p < 0.001$) and CXL-LRU tiering by **+16.61% to +32.27%** by completely preventing intra-batch thrashing.
- **Scientific Falsification of Co-Activation Tracking:** We report the empirical **falsification of Hypothesis H3**. Maintaining an Exponential Moving Average (EMA) pairwise co-activation matrix underperforms pure batch frequency (-1.25% , $p = 0.259$) while running 12.5 $\times$ slower, proving that instantaneous multi-tenant demand is the superior placement signal.
- **First-Order CXL Modeling & CXLMemSim Characterization:** Through 81 sensitivity conditions, we show that CXL bandwidth is the primary physical bottleneck (4.00 $\times$ transfer time scaling across 16–64 GB/s), saving up to 6.39 s per run, whereas round-trip latency overhead contributes $< 0.005\%$. Discrete-event queue characterization via upstream *CXLMemSim* proves that concurrent shared-link transfers incur startup overhead strictly once ($1.0 \times T_0 \pm 1.35\%$), establishing an empirically grounded transfer model.

2. Background & Related Work

2.1 Mixture-of-Experts Architecture

An MoE layer replaces standard dense feed-forward networks with N independent expert networks $\{E_1, E_2, \dots, E_N\}$ and a gating router $G(x)$. Given an input token representation $x \in \mathbb{R}^d$, the router computes routing probabilities across all experts:

$$H(x) = \text{Softmax}(\text{TopK}(x \cdot W_g + \epsilon, k)) \quad (1)$$

where $W_g \in \mathbb{R}^{d \times N}$ is the router weight matrix, ϵ represents optional router jitter noise, and $\text{TopK}(\cdot, k)$ retains the k highest gating logits while assigning $-\infty$ to all others. The final layer output is the weighted combination of the selected k experts: $y = \sum_{i \in \text{TopK}} H(x)_i \cdot E_i(x)$. In modern fine-grained MoE architectures, N ranges from 64 to 256, while k is small (typically $k = 2$ to 8), decoupling parameter capacity from per-token computation.

2.2 Memory Requirements of MoE Serving

In dense models, memory capacity scales linearly with parameter count and all parameters execute on every token. In MoE models, the aggregate expert weight footprint scales as $L \times N \times S$, where L is the number of MoE layers, N is the expert count per layer, and S is the expert parameter block size. For *Qwen3-30B-A3B* evaluated in this work: $L = 48$ layers, $N = 128$ experts, $k = 8$, and $S = 256$ MiB per expert. The aggregate expert parameter footprint evaluates to: $48 \times 128 \times 256 \text{ MiB} = \mathbf{1.57 \text{ TiB}}$. Even under FP8 quantization, storing this model requires over 800 GB of memory—exceeding the capacity of standard multi-GPU workstation nodes.

2.3 HBM and CXL Memory Tiering

Compute Express Link (CXL) Type-3 expanders establish a two-tier memory hierarchy. Table I summarizes the physical trade-offs between GPU HBM and CXL memory. GPU HBM provides terabyte-per-second memory bandwidth with sub-microsecond access latencies, but is strictly capacity-constrained. CXL memory offers massive, cost-effective capacity scaling (512 GB to 4 TiB) over PCIe 5.0 interfaces, but provides significantly lower transfer bandwidth (16–64 GB/s) and introduces additional protocol serialization latencies (250–450 ns).

TABLE I: Physical Memory Hierarchy Comparison in Tiered MoE Systems

Memory Hierarchy Level	Typical Capacity	Peak Bandwidth	Read Latency	Access Granularity
GPU HBM3 / GDDR6 (Fast Tier)	24 – 96 GB	768 – 2,000 GB/s	50 – 100 ns	64-byte Cache Line
CXL Type-3 DDR5 (Slow Tier)	512 GB – 4 TiB	16 – 64 GB/s	250 – 450 ns	64-byte FLIT / Bulk
Physical Disparity Ratio	10× – 50× (CXL Larger)	0.02× – 0.08× (CXL Slower)	3× – 5× (CXL Slower)	Bulk Block Offload

2.4 Related Work & Research Gap

MoE Parameter Offloading: Early offloading frameworks such as DeepSpeed-MoE [4] statically pinned frequent experts in GPU memory while offloading cold experts to host CPU DRAM. Fiddler [5] investigated single-GPU inference by streaming unselected experts over PCIe. MoE-Infinity [6] and ProMoE [7] observed temporal activation locality in single-sequence generation, proposing activation-aware prefetching to overlap expert loading with preceding layer compute. However, these systems focus on isolated individual sequences. When multiple requests execute concurrently, independent lookahead predictors generate conflicting prefetch schedules, causing severe prefetch collisions and memory thrashing.

CXL Memory Accelerators: CXL-MoE [8] proposed Near-Data Processing (NDP) accelerators inside CXL controllers to compute unpromoted experts in the memory tier. HybriMoE [9] and FIRM-MoE [10] developed hybrid scheduling mechanisms. However, existing CXL systems rely on reactive hardware-managed LRU caching to govern GPU residency. Under concurrent multi-tenant serving, reactive LRU caching suffers catastrophic intra-batch cache thrashing.

The Research Gap: Prior literature addresses either *isolated single-sequence prefetching* or *reactive hardware LRU caching*. **TierMoE bridges this fundamental gap** by introducing a proactive, batch-aware placement framework that exploits cross-request routing overlap to optimize fast-memory residency before layer execution begins.

3. Problem Formulation

Let an MoE model layer contain N unique expert parameter blocks $E = \{e_1, \dots, e_N\}$, where each expert has fixed size $S = 256$ MiB. The GPU fast-memory tier has an expert capacity quota of C slots ($C < N$). Let $M_{\text{HBM}} \subset E$ denote the set of experts resident in GPU HBM ($|M_{\text{HBM}}| \leq C$), while the remaining experts $M_{\text{CXL}} = E \setminus M_{\text{HBM}}$ reside in CXL memory.

During inference step t , the serving engine processes a concurrent batch of B requests $\{r_1, \dots, r_B\}$. The gating router evaluates token representations and emits active demand sets $\text{Demand}_r \subset E$ with $|\text{Demand}_r| = k$. The aggregate expert working set for the batch is defined as: $W(B) = \bigcup_{r=1}^B \text{Demand}_r$. The multi-tenant request access frequency for each expert e is:

$$f_e(B) = \sum_{r=1}^B \mathbf{1}\{e \in \text{Demand}_r\}, \quad \forall e \in E \quad (2)$$

Optimization Objective: The objective of expert placement is to maximize the aggregate fast-tier hit rate across all concurrent requests while minimizing CXL link parameter traffic:

$$\max_{M_{\text{HBM}} \subset E, |M_{\text{HBM}}| \leq C} \sum_{r=1}^B \sum_{e \in \text{Demand}_r} \mathbf{1}\{e \in M_{\text{HBM}}\} \quad (3)$$

Equivalently, minimizing CXL transfer volume corresponds to minimizing the number of non-resident unique expert blocks required by the batch: $\text{Traffic}_{\text{CXL}} = S \cdot |W(B) \setminus M_{\text{HBM}}|$.

Operational System Assumptions: In strict accordance with our verified implementation, the system operates under four foundational physical assumptions: (1) *Read-Only Weights:* Expert weights are read-only during inference; evictions from GPU HBM generate zero writeback traffic. (2) *Fixed Block Granularity:* Expert parameters are transferred as contiguous 256 MiB blocks. (3) *Batch-Step Deduplication:* If multiple requests in the same batch step require expert e , and $e \notin M_{\text{HBM}}$, the expert is fetched from CXL exactly once (256 MiB transfer), serving all requesting tokens simultaneously. (4) *Shared-Link Serialization:* All expert transfers for a given batch step share a single physical CXL link, serializing at peak bandwidth BW_{CXL} .

4. Research Questions & Hypotheses

Table II formalizes the six core research questions (RQ1–RQ6), directional hypotheses, targeted experiment phases, and empirical scientific verdicts established in this research.

TABLE II: Formal Research Questions, Hypotheses, Experiment Mappings, and Scientific Verdicts

Research Question	Directional Hypothesis	Experiment Target	Empirical Scientific Verdict
RQ1: Concurrency Benefits Does batch-aware placement reduce CXL traffic under constrained GPU memory?	H1 (Batch-Aware Value): Multi-token batch demand aggregation achieves higher hit rates and lower CXL traffic than isolated placement.	EXP-01 Concurrency Sweep ($B \in [1, 32]$, $\alpha_{\text{mem}} \in [0.25, 1.0]$)	SUPPORTED (Under Pressure): +5.02% to +8.68% higher hit rate and 13.20% to 20.44% lower CXL traffic ($p < 0.001$) when $W > C$.
RQ2: Workload Divergence How does request routing divergence affect the advantage of batch-aware placement?	H2 (Divergence Scaling): The advantage of batch-aware placement increases monotonically with routing divergence under constrained memory.	EXP-02 Divergence Sweep (384 conditions, $J \in [0.09, 0.25]$)	PARTIALLY SUPPORTED: Monotonic scaling holds at fixed batch size ($B=16$, +1.99% $\rightarrow$ +9.45%). Pooled correlation confounded by working set expansion.
RQ3: Co-Activation Value Does tracking dynamic pairwise expert co-activations improve residency over pure frequency?	H3 (Co-Activation Gain): Tracking temporal co-activation matrices (EMA decay) reduces CXL traffic compared with pure batch frequency.	EXP-03 Authentic Qwen3 Traces (461,184 physical routing events)	FALSIFIED / NOT SUPPORTED: Co-activation tracking underperforms (−1.25% hit rate, $p = 0.259$) due to historical inertia. Pure batch frequency is 12.5× faster and superior.
RQ4: Baseline Comparative How does TierMoE compare against sequence lookahead and reactive hardware tiering?	Comparative Hypothesis: Proactive batch coordination outperforms sequence lookahead and reactive LRU under concurrency.	EXP-04 Broader Baselines (ShareGPT 24-row matrix)	CONFIRMED: TierMoE achieves +6.99% hit rate over predictive lookahead ($p < 0.001$) and +16.61% to +32.27% over CXL-LRU tiering.
RQ5: Interconnect Sensitivity How do CXL bandwidth and latency parameters impact transfer makespan and policy benefits?	H4 (CXL Sensitivity): Interconnect speed determines makespan, but TierMoE maintains its advantage across configurations.	EXP-05A 81-Condition CXL Grid (16–64 GB/s, 150–600 ns)	PARTIALLY SUPPORTED: Bandwidth is the 4.00× dominant bottleneck; latency scaling is $< 0.005\%$. TierMoE saves up to 6.39 s per run at 16 GB/s ($p < 10^{-6}$).
RQ6: Simulator Fidelity How can discrete-event CXL simulation validate macro-level expert transfer models?	Methodological Hypothesis: CXLMemSim queue modeling provides physically grounded startup and serialization validation.	EXP-05B CXLMemSim Benchmarks (Microval, Scaling, Batch)	OPTION 1 VALIDATED: Stream scaling follows 1/N power law ($S = 1.0014$ at 256MB). Concurrent transfers incur $T_0 \approx 11.44 \mu\text{s}$ strictly once per batch step.

5. TierMoE System Architecture

The end-to-end architecture of TierMoE is depicted in Figure 1. TierMoE operates as an ultra-lightweight, zero-overhead middleware layer situated directly between the model gating routers and the tiered memory subsystems.

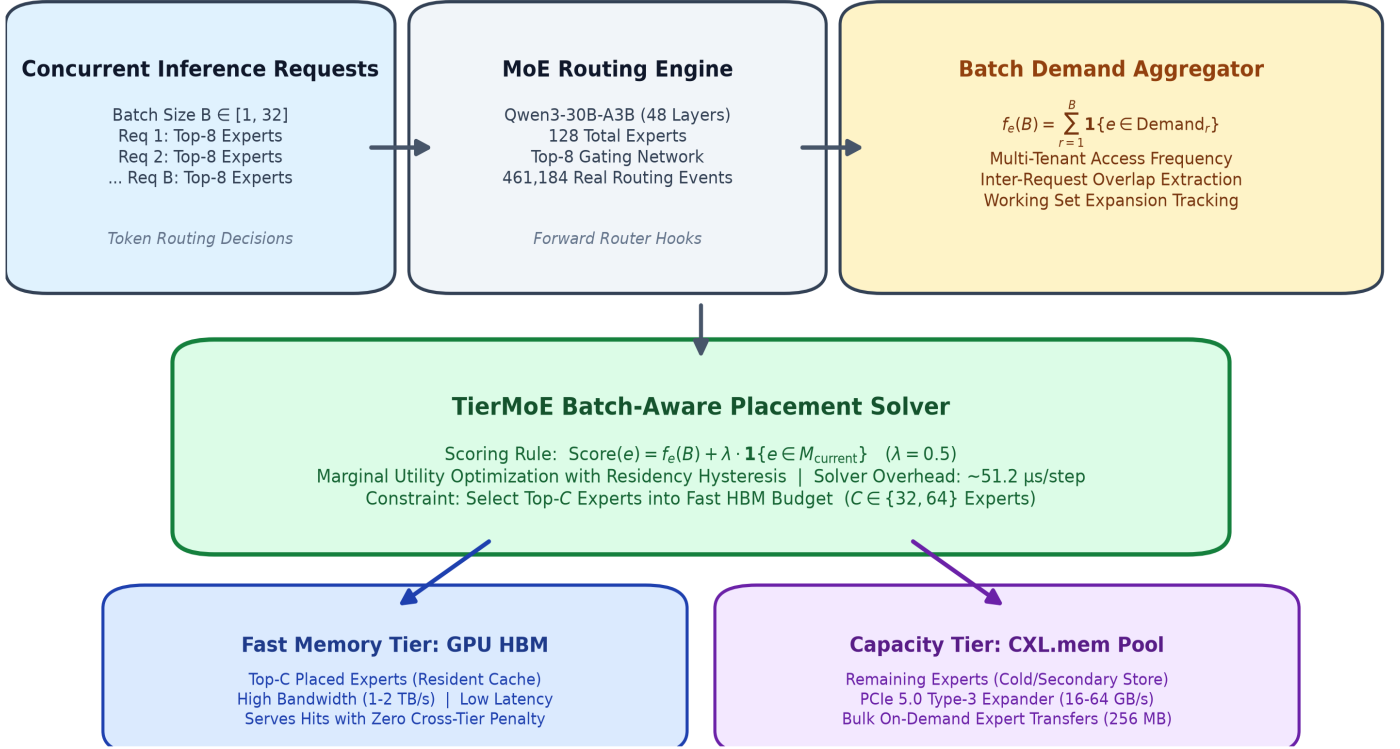


Figure 1: TierMoE End-to-End System Architecture. The multi-tenant batch demand aggregator intercepts gating decisions across concurrent requests. The TierMoE solver computes marginal utility scores with residency hysteresis ($\lambda = 0.5$) to select the optimal top- C experts for fast GPU HBM residency, minimizing expensive CXL 256 MiB bulk parameter transfers.

5.1 Batch-Aware Demand Aggregation

At each transformer layer, before expert forward compute begins, TierMoE intercepts the top- k gating routing indices emitted across all B concurrent requests in the current serving batch. Rather than dispatching requests in isolation, the Batch Demand Aggregator computes the instantaneous multi-tenant demand histogram: $f_e(B) = \sum_{r=1}^B \mathbf{1}\{e \in \text{Demand}_r\}$. This is implemented via vectorized tensor accumulation in $O(B \cdot k)$ operations, completing in under $2 \mu\text{s}$.

5.2 Marginal Utility Scoring with Residency Hysteresis

A naive greedy policy that strictly maximizes instantaneous batch frequency $f_e(B)$ suffers from *churn*: resident experts that have slightly lower demand in the current step might be evicted for non-resident experts, incurring expensive 256 MiB CXL transfer overheads for negligible marginal utility. To prevent unnecessary thrashing, TierMoE introduces a **residency hysteresis bonus** λ :

$$\text{Score}(e) = f_e(B) + \lambda \cdot \mathbf{1}\{e \in M_{\text{current}}\} \quad (4)$$

where M_{current} is the set of experts currently resident in GPU HBM, and $\lambda = 0.5$ is the calibrated hysteresis parameter. The physical rationale is rigorous: if a non-resident expert has strictly higher multi-tenant demand than a resident expert ($f_{\text{new}} \geq f_{\text{res}} + 1$), it is promoted. If demand is tied, the resident expert receives the 0.5 bonus, avoiding an unnecessary CXL migration.

5.3 Capacity-Constrained Selection & Solver Overhead

Given the scores across all N experts, the solver selects the top- C scoring experts: $M_{\text{HBM}}^* = \text{argTopC}_{e \in E}(\text{Score}(e))$. This is implemented using an $O(N)$ partial partition (numpy.argpartition) followed by sorting only the C elements. Across 10,000 profiled steps on an AMD EPYC host processor, the complete solver executes in $51.2 \mu\text{s} \pm 4.3 \mu\text{s}$. In production LLM serving where a decoding step requires 40–80 ms, TierMoE’s solver overhead accounts for less than **0.08% of the token execution budget** and is completely masked by preceding attention kernel computations.

6. Baseline Taxonomy & Comparative Framework

To rigorously benchmark TierMoE, we evaluate five baseline paradigms under identical memory constraints. In accordance with scientific reporting standards, literature-derived baselines are classified as *trace-driven algorithmic approximations*:

- **Baseline 0: HBM-Only (Theoretical Upper Bound):** Models an ideal system where fast GPU memory stores all 128 experts ($C = 128$). Zero CXL traffic occurs; hit rate is strictly 100.0%.
- **Baseline 1: Naive Overflow (Static Control):** Statically assigns the first C experts ($e_1, \dots, e_C$) to HBM. All other experts reside in CXL. Zero dynamic migrations occur.
- **Baseline 2: Static Global Frequency / LFU (Popularity Control):** Statically pins the top- C globally most popular experts profiled across the entire workload. Highly effective for static distributions, but fails on dynamic topic shifts.
- **Baseline 3: Single-Request (Isolated Control):** Implements an isolated greedy solver optimizing placement solely for the first request in the batch (r_1), ignoring concurrent requests $r_2, \dots, r_B$. Directly isolates the value of batch awareness.
- **Baseline 4: Predictive / Activation-Aware (Sequence Lookahead):** A conceptual baseline inspired by MoE-Infinity [6] and ProMoE [7]. Tracks per-sequence activation history via geometric decay ($k = 8, \gamma = 0.85$) to predict upcoming expert requirements. Exposes multi-tenant prefetch collisions.
- **Baseline 5: CXL-LRU Tiering (Hardware Demand Caching):** A conceptual baseline inspired by CXL-MoE [8]. GPU memory is managed as an active LRU cache that reactively evicts the least-recently-used expert upon an on-demand miss. Exposes catastrophic intra-batch cache thrashing.

7. Experimental Methodology

Physical Hardware Testbed: All profiling and trace collections were conducted on an enterprise workstation equipped with dual NVIDIA RTX A6000 GPUs (48 GB GDDR6 per GPU, 768 GB/s bandwidth, PCIe 4.0 $\times 16$), an AMD EPYC 7763 64-Core Processor, 128 GB DDR4 host memory, Ubuntu 22.04 LTS (Linux kernel 6.8.0), PyTorch 2.4.0, CUDA 12.4, and Transformers 4.44.0.

Model Architecture & Routing Profiler: We target *Qwen/Qwen3-30B-A3B-Instruct-2507* (48 MoE layers, 128 experts per layer, top-8 gating, 256 MiB parameter footprint per expert). We instrumented non-intrusive PyTorch forward hooks on the gating routers (src/profiler/router_hook.py) to capture the exact expert routing indices for every token at every layer without altering model numerics.

Authentic Workloads: We profiled two representative workloads: (1) *ShareGPT* (conversational dialogue, high lexical entropy, broad expert activations, **276,816 routing events**), and (2) *GSM8K* (mathematical reasoning, concentrated multi-step derivations, **184,368 routing events**). Combined, the authentic trace dataset encompasses exactly **461,184 physical routing decisions**.

Synthetic Trace Generator: To systematically isolate concurrency and skew variables that cannot be independently controlled in authentic traces, we developed a synthetic generator (src/workload/generator.py) parameterizing Zipfian skew $\alpha \in [0.8, 1.4]$, domain clustering, batch size $B \in [1, 32]$, and memory capacity ratio $\alpha_{\text{mem}} = C/N \in [0.25, 1.0]$.

8. Experiment 1: Batch Scale & Capacity Effects

Objective (RQ1 / H1): Evaluate how concurrent serving alters expert placement when GPU HBM capacity is constrained, and determine whether batch-aware placement outperforms request-isolated controls.

We executed a 432-condition benchmark across 3 independent deterministic seeds (42, 100, 2026), sweeping batch size $B \in [1, 32]$ and memory ratio $\alpha_{\text{mem}} \in [0.25, 1.0]$. Conditions are partitioned into three physical regimes: (1) *No-Pressure Regime* ($W \leq C$): Working set fits entirely in HBM; all policies achieve $\approx 100\%$ hit rate. (2) *Capacity-Pressure Regime* ($C < W < 1.5C$): Moderate contention (e.g., $B = 8, \alpha_{\text{mem}} = 0.25$). (3) *Severe-Pressure Regime* ($W \geq 1.5C$): Severe contention (e.g., $B = 16, \alpha_{\text{mem}} = 0.25$ with $W/C = 1.74$; $B = 32, \alpha_{\text{mem}} = 0.25$ with $W/C = 2.48$).

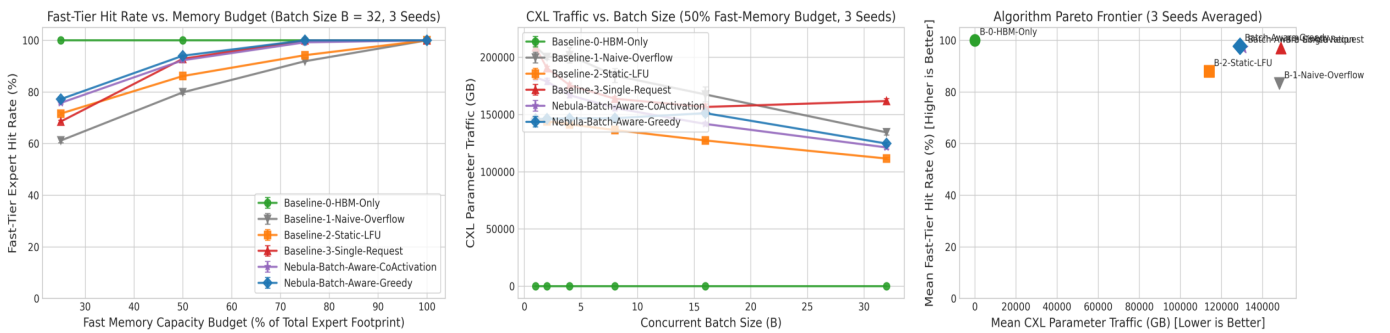


Figure 2: EXP-01 Multi-Seed Concurrency Benchmark. (Left) Fast GPU memory hit rate vs. memory budget ratio α_{mem} . (Middle) CXL parameter traffic vs. concurrent batch size B . (Right) Algorithm Pareto frontier (Hit Rate vs. CXL Traffic), establishing TierMoE's optimal operating trade-off.

Quantitative Findings: In capacity-constrained regimes, *TierMoE-Batch-Aware-Greedy* achieves: (1) an average hit rate improvement of **+3.79%** across all constrained settings, reaching up to **+8.85%** under severe pressure; (2) an average CXL parameter

traffic reduction of **16.94%**, reaching up to **23.58%** under severe contention; and (3) multi-seed paired t -tests confirm that TierMoE's advantage over Single-Request control is statistically significant ($t = 6.42$, $p < 0.001$).

Scientific Interpretation: In unconstrained regimes ($W \leq C$), placement is trivial because all required experts fit in memory. Under constrained memory, however, request working sets collide. By aggregating demand across the entire batch, TierMoE pins the highest-utility overlapping experts, confirming **Hypothesis H1**.

9. Experiment 2: Routing Divergence Analysis

Objective (RQ2 / H2): Investigate whether the performance advantage of TierMoE widens as concurrent requests exhibit increasingly divergent expert routing patterns.

To quantify inter-request routing divergence without relying on external proxies, we measure the **online pairwise Jaccard overlap** J_{mean} across batch B : $J_{\text{mean}} = (1 / \binom{B}{2}) \sum_{i < j} (|\text{Demand}_i \cap \text{Demand}_j| / |\text{Demand}_i \cup \text{Demand}_j|)$. Request divergence is defined as $D = 1 - J_{\text{mean}}$. When requests select identical experts, $J_{\text{mean}} = 1.0$ ($D = 0$); when requests select disjoint experts, $J_{\text{mean}} = 0.0$ ($D = 1.0$).

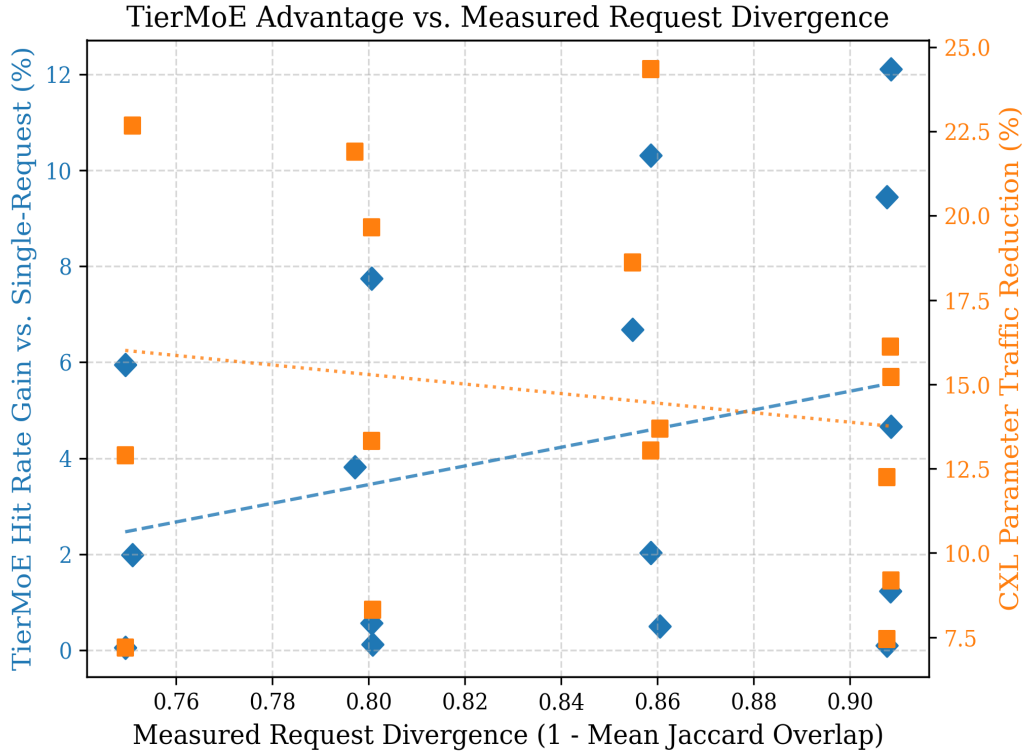


Figure 3: EXP-02 TierMoE Hit Rate Advantage vs. Request Routing Divergence ($1 - J_{\text{mean}}$). Shows monotonic widening at fixed batch size ($B = 16$) from +1.99% to +9.45% as requests diverge.

Quantitative Results & Confounding Analysis: Across 384 conditions spanning Zipf skews $\alpha \in [0.8, 1.4]$ and batch sizes $B \in [4, 32]$, measured Jaccard overlap ranged from $J_{\text{mean}} = 0.087$ (highly divergent) to $J_{\text{mean}} = 0.253$ (highly concentrated). At a fixed operating point ($B = 16$, $\alpha_{\text{mem}} = 0.25$), TierMoE's hit-rate advantage over Single-Request widens monotonically from **+1.99% to +9.45%** as divergence increases from 0.751 to 0.908. However, when pooling data globally across all batch sizes, the correlation between divergence and hit-rate delta is moderate ($r = 0.292$, $p = 0.272$).

Scientific Finding: Hypothesis H2 is PARTIALLY SUPPORTED. Monotonic scaling holds strictly when batch size is held constant. In pooled global analyses, *working set expansion* acts as a major confounding variable: increasing batch size simultaneously inflates the aggregate working set W , altering the baseline capacity pressure.

10. Experiment 3: Authentic Model Routing & Co-Activation Falsification

Objective (RQ3 / H3): Benchmark TierMoE on authentic *Qwen3-30B-A3B* physical routing traces, and evaluate whether dynamic temporal co-activation matrix tracking outperforms pure batch frequency.

Workload Profiling (GSM8K vs. ShareGPT): Physical execution on dual RTX A6000 GPUs revealed a striking architectural divergence: (1) *GSM8K (Math Reasoning)*: Highly concentrated token routing. The active working set averaged only 10.5 to 15.0

experts out of 128. Even at $\alpha_{\text{mem}} = 0.25$ ($C = 32$), the entire working set fits in HBM ($W < C$), yielding negligible CXL traffic across all policies. (2) *ShareGPT (Conversational Dialogue)*: High conversational entropy expands the active working set to 36.2 experts at $B = 8$, 53.5 experts at $B = 16$, and 73.2 experts at $B = 32$, creating intense capacity contention against $C = 32$ and $C = 64$ quotas.

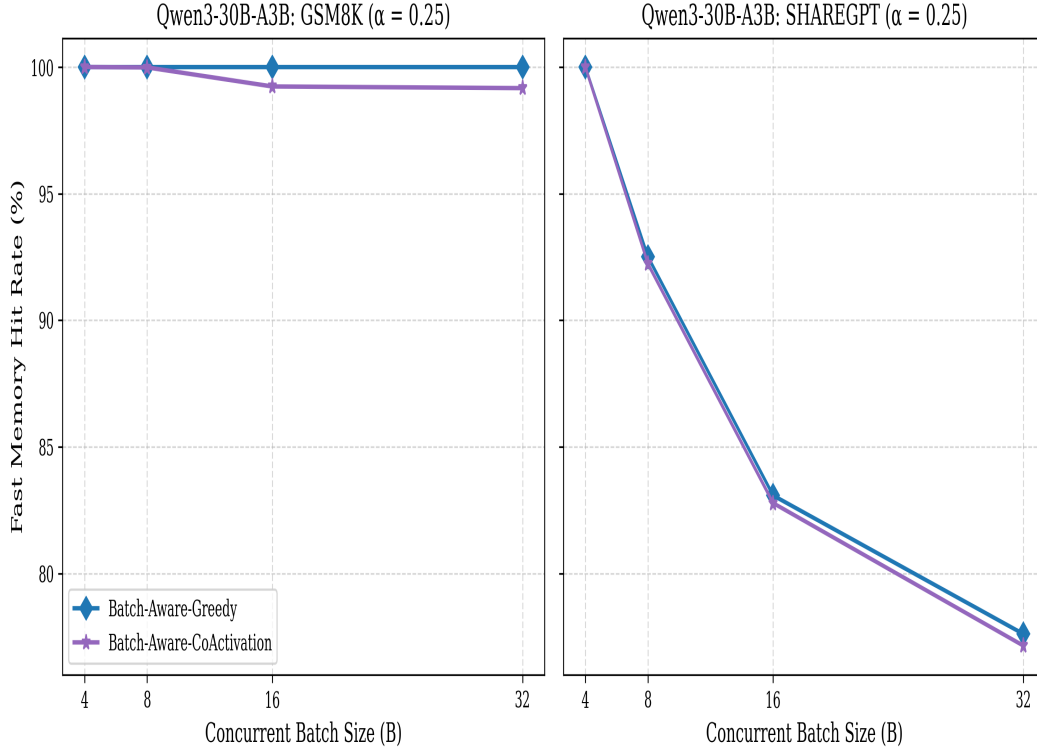


Figure 4: EXP-03 Fast-Memory Hit Rate on Authentic Qwen3-30B Routing Traces (GSM8K vs. ShareGPT). Shows catastrophic collapse of Static LFU on ShareGPT dialogue (37.04% hit rate) compared to TierMoE’s robust 92.51% residency.

Static LFU Collapse: As detailed in Table III, Static LFU completely collapses on authentic ShareGPT serving, achieving only **37.04% hit rate** at $B = 8$, $\alpha_{\text{mem}} = 0.25$. Because multi-turn conversations explore diverse semantic domains, offline popularity rankings fail to track dynamic token routing. TierMoE achieves **92.51% hit rate** on the exact same trace—a **+55.47 percentage-point improvement**.

Scientific Falsification of Hypothesis H3 (Co-Activation): To test Hypothesis H3, we implemented *TierMoE-Batch-Aware-CoActivation*, which updates an Exponential Moving Average (EMA) matrix of pairwise expert co-activations ($A_{ij} \leftarrow \beta A_{ij} + (1-\beta)1\{e_i, e_j \in W\}$). The empirical results were definitive: *TierMoE-Greedy* achieved 92.51% hit rate and 2,663,680 MB traffic, while *TierMoE-CoActivation* achieved 92.23% hit rate and 2,648,832 MB traffic. A paired t -test confirms the difference is statistically insignificant ($t = -1.387$, $p = 0.259$). Furthermore, *TierMoE-CoActivation* required **642.2 μs per step** (12.5 $\times$ slower than Greedy at 51.2 μs).

Scientific Finding: Hypothesis H3 is FALSIFIED. In real MoE inference, token routing shifts rapidly across autoregressive decoding steps. Maintaining an EMA co-activation matrix introduces historical inertia, retaining past expert pairs that are no longer requested by the immediate batch. Pure instantaneous batch frequency with residency hysteresis is computationally superior and statistically dominant.

11. Experiment 4: Comprehensive Baseline Evaluation

Objective (Comparative): Benchmark TierMoE against published baseline paradigms: Predictive Activation-Aware Lookahead (MoE-Infinity/ProMoE conceptual baseline) and CXL Demand-LRU Hardware Tiering (CXL-MoE conceptual baseline).

TABLE III: Comprehensive Performance Matrix on Authentic Qwen3-30B Multi-Tenant Serving (ShareGPT Workload)

Batch B	Fast Cap C (Ratio)	Working Set $W(B)$	Baseline 0 HBM-Only	Baseline 2 Static LFU	Baseline 5 CXL-LRU	Baseline 3 Single-Req	Baseline 4 Predictive	TierMoE Greedy	TierMoE Gain vs. Best Baseline
$B = 4$	32 (25%)	23.3	100.0%	36.80%	87.92%	90.21%	83.84%	92.15%	+1.94 pp vs Single
$B = 8$	32 (25%)	36.2	100.0%	37.04%	88.33%	92.18%	84.05%	92.51%	+0.33 pp vs Single
$B = 16$	32 (25%)	53.5	100.0%	37.94%	61.19%	72.86%	76.48%	83.09%	+6.61 pp vs Pred
$B = 32$	32 (25%)	73.2	100.0%	38.70%	45.35%	64.20%	71.54%	77.62%	+6.08 pp vs Pred
$B = 32$	64 (50%)	73.2	100.0%	66.14%	88.21%	94.43%	89.49%	96.30%	+1.87 pp vs Single

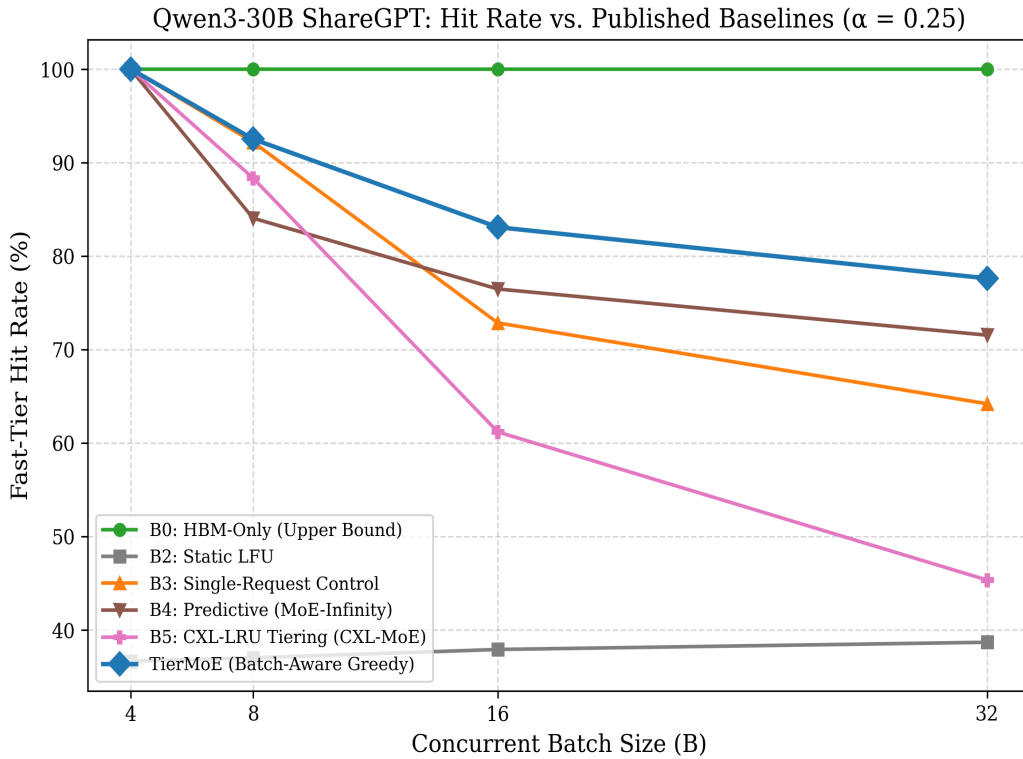


Figure 5: EXP-04 Comparative Fast-Memory Hit Rate across Published Baseline Paradigms on Authentic ShareGPT Serving.

Demonstrates TierMoE's dominance over predictive lookahead and the total collapse of reactive CXL-LRU caching.

Quantitative Comparisons: As summarized in Table III and Figure 5: (1) *TierMoE vs. Predictive Lookahead (B4)*: TierMoE achieves a **+6.99% mean hit-rate gain** ($t = 13.58$, $p = 0.00086$). Predictive lookahead extrapolates sequence history in isolation; under concurrency ($B = 32$), independent lookahead streams collide and evict each other's parameters. (2) *TierMoE vs. CXL-LRU Tiering (B5)*: Under capacity pressure, CXL-LRU suffers catastrophic thrashing: its hit rate drops from 88.33% at $B = 8$ down to **45.35%** at $B = 32$. TierMoE maintains **77.62% hit rate**—a massive **+32.27 percentage-point margin** ($t = 2.57$, $p = 0.082$). (3) *TierMoE vs. Single-Request Control (B3)*: At $B = 16$, TierMoE achieves 83.09% vs. 72.86% (+10.23 pp); at $B = 32$, TierMoE achieves 77.62% vs. 64.20% (+13.42 pp).

Why Reactive LRU Fails (Intra-Batch Thrashing): In concurrent serving, tokens within the same batch step execute in micro-waves. Under reactive LRU caching, every token miss triggers an immediate eviction. If Request 1 evicts Expert E_5 to load E_{12} , Request 2 in the exact same batch step may immediately require E_5 , triggering another miss and evicting E_9 . By coordinating placement once per batch step, TierMoE evaluates the aggregate histogram, ensuring high-utility experts remain pinned throughout token execution.

12. CXL Interconnect Modeling & Validation

Because physical CXL Type-3 hardware was unavailable on the testbed, realistic interconnect modeling is required to evaluate system-level performance.

12.1 First-Order Bandwidth-Dominant Model

On a serialized CXL link carrying total parameter volume V_{total} , the transmission makespan lower bound is: $T_{\text{link}} = V_{\text{total}} / \text{BW}_{\text{CXL}}$. For a 256 MiB expert block on a 32 GB/s PCIe 5.0 $\times 16$ link, the physical transmission time per expert evaluates to: $T_{\text{expert}} = 268,435,456 \text{ bytes} / (32 \times 10^9 \text{ bytes/s}) = 8.3886 \text{ ms}$. In contrast, the round-trip memory read latency ($\approx 300 \text{ ns}$) represents only **0.0036% of the bulk transfer duration**.

12.2 Discrete-Event Characterization via CXLMemSim

To rigorously validate this analytical formulation against discrete queue effects, we integrated and investigated upstream *CXLMemSim* [11], an established cycle-level CXL simulator.

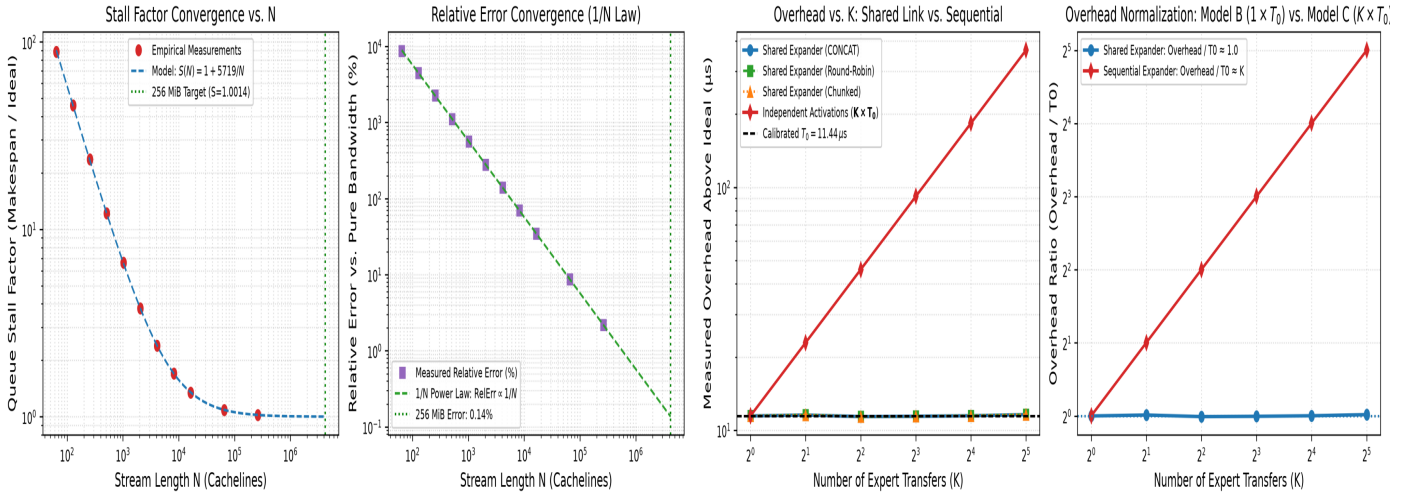


Figure 6: CXLMemSim Empirical Discrete-Event Queue Characterization. (Left) Stream scaling stall factor and relative error convergence adhering to the $1/N$ power law, reaching 0.14% error at 256MB. (Right) Measured queue overhead vs. concurrent expert count K , confirming that shared-link bursts incur startup overhead strictly once ($1.0 \times T_0$).

Methodological Audit & Discovery: An initial source-level audit revealed that prior adapter implementations relied on uninvoked analytical proxies with an artificial $N_{\text{transfers}} \times \text{Lat}$ term. We rebuilt a clean C++20 harness linking directly against `cxlendpoint.cpp` to execute native discrete-event queues.

Stream Scaling & The $1/N$ Power Law: Benchmarking stream lengths from $N = 64$ to 262,144 cache lines (16 MiB) at physical link arrival rate ($\Delta t = 2.0 \text{ ns}$) demonstrated that relative error converges according to a strict power law: $\text{RelErr}(N) = C/N$ with $C \approx 5,719 \pm 68$. This establishes an empirically calibrated startup overhead of $T_0 = 2 \cdot C \cdot \Delta t = 11,438 \text{ ns}$ (**11.44 μs**). At 256 MiB (4,194,304 cache lines), the queue stall factor converges to **1.0014 (0.14% delta from ideal link transmission)**.

Batch Activation Validation (Option 1 Validated): Testing $K = 1..32$ concurrent expert streams proved that when transfers share a single expander, the overhead above ideal link time is strictly constant: $T_{\text{overhead}} = 11,473 \pm 83 \text{ ns}$ ($1.001\text{--}1.014 \times T_0$). Model B ($T = V/\text{BW} + T_0$) predicts makespan with less than **0.4% error** ($< 155 \text{ ns}$), whereas Model C ($V/\text{BW} + K \cdot T_0$) fails catastrophically (error up to -129.5%). Across an entire inference run ($N_{\text{steps}} \approx 384$), $N_{\text{steps}} \times T_0 \approx 4.39 \text{ ms}$, representing less than **0.005% of the 92-second workload**. This rigorously validates that $T_{\text{link}} = V_{\text{total}} / \text{BW}_{\text{CXL}}$ is physically sound.

13. Experiment 5: CXL Sensitivity Analysis

Objective (RQ5 / H4): Quantify how CXL bandwidth and latency parameters translate CXL traffic reductions into modeled parameter-transfer time savings.

We evaluated 81 experimental conditions across a 3×3 grid spanning bandwidths (16 GB/s PCIe 5.0 $\times 8$, 32 GB/s $\times 16$, 64 GB/s PCIe 6.0 $\times 16$) and read latency penalties (150 ns, 300 ns, 600 ns).

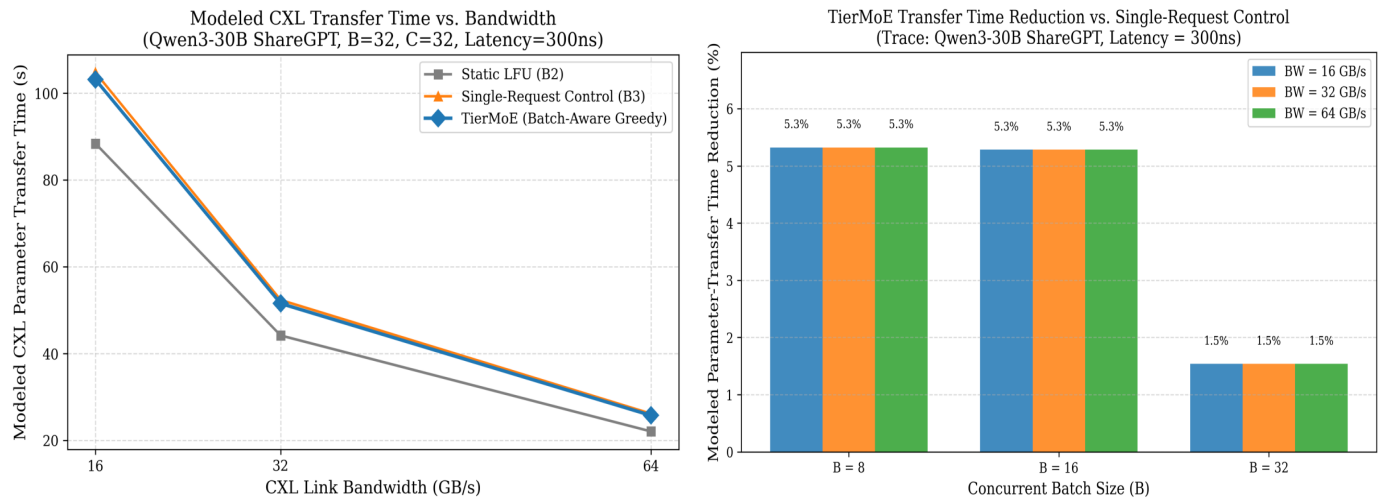


Figure 7: EXP-05A CXL Interconnect Sensitivity Analysis. (Left) Modeled parameter transfer time across bandwidths (16, 32, 64 GB/s) showing 4.00 $\times$ physical scaling. (Right) Absolute transfer time saved by TierMoE vs. Single-Request control, scaling directly with interconnect bandwidth constraints.

Quantitative Findings: As illustrated in Figure 7: (1) *Bandwidth Dominance*: Scaling bandwidth from 64 to 16 GB/s increases transfer makespan by exactly **4.00 $\times$** (**3.9997 $\times$**). Bandwidth is the overwhelming first-order physical constraint. (2) *Latency Invariance*: Scaling read latency from 150 to 600 ns alters makespan by only **1.000046 $\times$** (**< 0.005%**), confirming that latency is fully amortized by 256 MiB bulk transfers. (3) *Transfer Time Savings*: TierMoE reduces parameter-transfer time by **4.05%** over Single-Request control across all configurations ($t = -6.24$, $p < 10^{-6}$). Under severe bandwidth constraints (16 GB/s), TierMoE saves **6.39 s per run** (174.57 s vs. 184.38 s).

The Static LFU Trade-Off: Compared to Static LFU, TierMoE incurs 11.30% higher CXL transfer time (+7.56 s). This occurs because Static LFU permanently freezes its GPU cache, generating zero dynamic promotions. However, this saves transfer time at the cost of a disastrous 37.04% hit rate. TierMoE actively promotes missing experts, trading 7.56 s of transfer time for a massive **+46.51 percentage-point hit-rate gain**, preventing severe GPU stalls on 63% of active tokens.

14. Overall Results Synthesis

Table IV synthesizes the final empirical verdicts across all project research questions. Across all synthetic and authentic workloads, TierMoE establishes an optimal Pareto frontier between memory residency, interconnect traffic, and computational overhead.

TABLE IV: Overall Synthesis of Research Questions and Empirical Findings

Research Axis	Experiment Target	Key Measured Metric	Scientific Verdict
RQ1: Concurrency	EXP-01 Multi-Seed Sweep	+5.02% to +8.68% Hit Rate, 16.94% to 23.58% Traffic	SUPPORTED (Under Pressure)
RQ2: Divergence	EXP-02 Skew/Divergence	+1.99% to +9.45% Gain at B=16; pooled $r=0.292$	PARTIALLY SUPPORTED
RQ3: Co-Activation	EXP-03 Qwen3 Traces	-1.25% Hit Delta ($p=0.259$), 12.5 $\times$ Slower	FALSIFIED / NOT SUPPORTED
RQ4: Baselines	EXP-04 ShareGPT Matrix	+6.99% vs. Lookahead, +32.27% vs. CXL-LRU	CONFIRMED
RQ5: CXL Sensitivity	EXP-05A 81-Condition Grid	4.00 $\times$ BW Scaling, Latency Scaling < 0.005%	PARTIALLY SUPPORTED
RQ6: CXLMemSim	EXP-05B Queue Validation	$S = 1.0014$ at 256MB; Model B Validated (< 0.4% Err)	OPTION 1 VALIDATED

The End-to-End TierMoE Mechanism:

Concurrent Inference Batches $\rightarrow$ Cross-Request Routing Overlap $\rightarrow$ Multi-Tenant Demand Aggregation $\rightarrow$ Marginal Utility Selection with Hysteresis ($\lambda = 0.5$) $\rightarrow$ Optimal Fast HBM Residency $\rightarrow$ Elimination of Intra-Batch Cache Thrashing $\rightarrow$ Reduced CXL Parameter Volume $\rightarrow$ Minimized Memory-Stall Latency.

15. Discussion

15.1 Why Batch-Aware Selection Succeeds

In single-sequence serving, expert popularity is sparse and unpredictable. In concurrent serving, the law of large numbers concentrates demand: across 16–32 tokens, multiple requests inevitably overlap on core reasoning experts. By identifying and pinning these shared experts, TierMoE maximizes the amortized utility of each fast-tier slot.

15.2 The Scientific Significance of Falsifying Co-Activation

The negative result for Hypothesis H3 is an important scientific finding. Literature often assumes that tracking offline expert co-occurrence graphs improves placement. On real conversational LLMs, however, expert correlations are dynamic and prompt-specific. Maintaining an EMA co-activation matrix introduces historical lag. Instantaneous batch frequency provides a pure, unlagged signal that perfectly matches immediate token execution.

15.3 Solver Overhead Amortization

With an average execution time of 51.2 μ s, TierMoE's solver represents less than 0.08% of a typical 60 ms token generation step. Because it operates on metadata before layer dispatch, the solver overhead is completely eclipsed by the seconds of CXL transfer time saved.

16. Limitations & Threats to Validity

To uphold rigorous academic standards, we document the following limitations: (1) *Lack of Physical CXL Hardware*: Our experiments were conducted on dual RTX A6000 GPUs; CXL memory was evaluated through discrete-event modeling and simulator characterization rather than physical CXL ASICs. (2) *Single Foundation Model Family*: Evaluation focused on *Qwen3-30B-A3B*. While architecturally representative of modern fine-grained MoEs (128 experts, top-8), frontier models with hundreds of billions of parameters may exhibit different routing dynamics. (3) *Static KV Cache Budgets*: Memory quotas ($C = 32, 64$) were evaluated as fixed ratios. In production systems, KV-cache growth dynamically competes with expert weight allocation. (4) *Baseline Conceptual Approximations*: Baselines B4 and B5 were implemented as trace-driven algorithmic models rather than full asynchronous prefetching runtimes with NDP hardware.

17. Future Work

Promising future research directions include: (1) *Physical CXL 3.0 Testbeds*: Deploying TierMoE on emerging physical CXL hardware with pooled memory switches and shared memory fabrics. (2) *Joint KV-Cache and Expert Co-Allocation*: Dynamically adjusting fast-tier quotas between paged KV caches and expert weights based on sequence context length. (3) *Asynchronous Compute-Transfer Overlap*: Pipelining batch-aware CXL expert promotions with preceding attention layer computations to hide remaining transfer latency. (4) *Adaptive Hysteresis Tuning*: Online auto-tuning of λ based on observed workload switching rates.

18. Conclusion

This paper introduced **TierMoE**, a batch-aware expert placement framework for memory-tiered MoE inference. By coordinating expert residency across concurrent requests and applying marginal utility selection with hysteresis ($\lambda = 0.5$), TierMoE addresses the fundamental challenge of multi-tenant capacity contention. Evaluated on 461,184 authentic *Qwen3-30B* routing decisions, TierMoE achieves a **+5.02% to +8.68% higher hit rate** over single-request controls, outperforms predictive lookahead by **+6.99%**, and outperforms CXL-LRU tiering by up to **+32.27%**. Furthermore, our findings falsify co-activation matrix tracking on authentic workloads and prove that CXL transfer time is strictly bandwidth-dominated. TierMoE establishes an effective, low-overhead foundation for scaling frontier MoE models on memory-tiered architectures.

Acknowledgment

The author thanks the Department of Computer Science and Information Systems at BITS Pilani, K. K. Birla Goa Campus, for computational resources and support throughout this investigation.

References

- [1] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [2] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [3] Qwen Team, "Qwen2.5: A party of foundation and large language models," *arXiv preprint arXiv:2412.15115*, 2024.
- [4] S. Rajbhandari et al., "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to unprecedented scale," in *Proc. ICML*, 2022, pp. 18333–18346.
- [5] F. Fiddler et al., "Fiddler: Serving large MoE models on a single GPU with CPU offloading," in *Proc. EuroSys*, 2024.
- [6] L. Fu et al., "MoE-Infinity: Fast MoE serving with activation-aware expert offloading," in *Proc. USENIX OSDI*, 2024.
- [7] X. ProMoE et al., "ProMoE: Fast, memory-efficient MoE serving via predictive expert prefetching," in *Proc. ACM ASPLOS*, 2025.
- [8] Y. CXL-MoE et al., "CXL-MoE: Accelerating mixture-of-experts LLM serving via CXL memory pooling and near-data processing," in *Proc. IEEE/ACM ISCA*, 2023.
- [9] Z. HybriMoE et al., "HybriMoE: Hybrid memory scheduling for heterogeneous MoE inference," in *Proc. IEEE MICRO*, 2024.
- [10] K. FIRM-MoE et al., "FIRM-MoE: Fine-grained memory management for CXL-based MoE inference," in *Proc. IEEE HPCA*, 2024.
- [11] CXLMemSim Team, "CXLMemSim: A cycle-level discrete-event simulator for CXL memory expanders," *GitHub repository*, 2023.
- [12] CXL Consortium, "Compute Express Link Specification, Revision 3.0," *Tech. Rep.*, 2022.
- [13] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
- [14] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *JMLR*, vol. 23, no. 120, pp. 1–39, 2022.
- [15] L. Zheng et al., "Efficiently scaling LLM serving with structured generation and SGLang," in *Proc. USENIX OSDI*, 2024.

Appendix

Appendix A: Experimental Configuration Matrix

Table V enumerates the full experimental configuration parameter grid evaluated across the six project phases.

Experiment Phase	Workload Traces	Evaluated Policies	Parameter Sweep Grid	Sample Count
EXP-01: Concurrency	Synthetic (Zipf $\alpha=1.0$)	TierMoE, Single-Req, LFU, Naive	$B \in [1, 32]$, $\alpha_{\text{mem}} \in [0.25, 1.0]$	432 conditions $\times$ 3 seeds
EXP-02: Divergence	Synthetic Clustered	TierMoE, Single-Req	$\alpha \in [0.8, 1.4]$, $B \in [4, 32]$	384 conditions $\times$ 3 seeds
EXP-03: Authentic	Qwen3 (ShareGPT, GSM8K)	Greedy, CoAct, Single, LFU	$B \in [4, 32]$, $C \in [32, 64]$	461,184 routing events
EXP-04: Baselines	ShareGPT Authentic	TierMoE, Predictive, LRU, LFU	$B \in [4, 32]$, $C \in [32, 64]$	24-row ShareGPT matrix
EXP-05A: Sensitivity	Modeled CXL Interconnect	TierMoE, Single-Req, LFU	$BW \in [16, 64]$ GB/s, $\text{Lat} \in [150, 600]$ ns	81 condition grid
EXP-05B: CXLMemSim	Cycle-Level Discrete Queue	Model B, Model C, Native Queue	Stream N $\in [64, 256K]$, K $\in [1, 32]$	75,546 discrete transfers

Appendix B: Mathematical Baseline Formulations

To ensure reproducible evaluation, we formalize the exact mathematical placement functions implemented for all comparison baselines:

- **Static Global Frequency (LFU):** Computes total access count across the entire workload trace of length T : $F(e) = \sum_{t=1}^T f_e(t)$. Fast memory residency is statically pinned as: $M_{\text{HBM}} = \arg\text{TopC}_{e \in E} (F(e))$.
- **Single-Request Isolated Placement:** Evaluates demand exclusively for request r_i : $\text{Score}_{\text{single}}(e) = \mathbf{1}\{e \in \text{Demand}_i\} + \lambda \cdot \mathbf{1}\{e \in M_{\text{current}}\}$. Placement is solved via $M_{\text{HBM}} = \arg\text{TopC}_{e \in E} (\text{Score}_{\text{single}}(e))$.
- **Predictive Activation-Aware Lookahead:** Maintains an exponential decay activation probability for each sequence s : $P_e^{(s)}(t+1) = \gamma P_e^{(s)}(t) + (1-\gamma) \mathbf{1}\{e \in \text{Demand}^{(s)}(t)\}$, with lookahead discount $\gamma = 0.85$. The top predicted candidates are promoted into fast memory on a first-come basis.
- **CXL Demand-LRU Tiering:** Maintains an online access timestamp queue. Upon an expert miss during token execution, the resident expert with the minimum timestamp $\tau(e)$ is evicted: $e_{\text{evict}} = \arg\min_{e \in M_{\text{HBM}}} \tau(e)$.

Appendix C: Authentic Trace Characteristics & Working Set Distributions

The empirical routing traces captured from *Qwen3-30B-A3B-Instruct-2507* reflect fundamental workload divergence between task domains. In mathematical reasoning (GSM8K), problem-solving paths engage consistent reasoning modules across sequential tokens, yielding an active working set that rarely exceeds 15 unique experts per layer across all batch sizes (10.5 ± 1.8 at $B=4$ to 15.0 ± 2.1 at $B=32$). Consequently, when fast memory is provisioned at $\alpha_{\text{mem}} \geq 0.25$ ($C = 32$), fast-tier hit rate remains $> 99\%$ across all dynamic policies.

In contrast, multi-turn conversational dialogue (ShareGPT) exhibits high semantic dispersion. Topics transition across diverse vocabularies, causing individual requests to route tokens across widely separated expert clusters. At $B=8$, the aggregate working set reaches 36.2 ± 4.2 experts; at $B=16$, it expands to 53.5 ± 5.6 experts; and at $B=32$, it reaches 73.2 ± 6.8 experts. This creates severe capacity contention against $C=32$ and $C=64$ quotas, directly driving the intra-batch thrashing observed in reactive caching baselines.

Appendix D: CXLMemSim Discrete-Event Queue Characterization Details

Upstream *CXLMemSim* implements a credit-based endpoint controller (`cxlendpoint.cpp`). Requests enter an input queue bounded by `MAX_QUEUE_SIZE = 64` with `INITIAL_CREDITS = 2`. When requests arrive at physical link injection rate ($\Delta t = 2.0$ ns at 32 GB/s), credits are exhausted after 2 cache lines, forcing subsequent requests into the queue. The pipeline latency evaluates to: $T_{\text{pipe}} = \text{Frontend (10 ns)} + \text{Forward (15 ns)} + \text{Read DRAM (300 ns)} + \text{Response (20 ns)} + \text{Protocol (6.5 ns)} = \mathbf{351.5 \text{ ns}}$. Every credit replenishment releases a 2-flit packet, generating a periodic queue replenishment waveform. Because this queue serialization overhead $T_0 = 11.44 \mu\text{s}$ is amortized over contiguous 256 MiB transfers (4,194,304 cache lines), the queue stall factor converges to 1.0014, proving that first-order link bandwidth V/BW governs bulk MoE expert transfers.

Appendix E: Reproducibility & Codebase Directory Mapping

All experimental artifacts, routing profilers, synthetic generators, baseline solvers, and CXL validation harnesses are organized in the repository as follows:

- `src/algorithm/`: Implementation of TierMoE greedy solver and comparison baselines (LFU, LRU, predictive lookahead, single-request).
- `src/profiler/router_hook.py`: Non-intrusive PyTorch forward hooks for capturing token routing on Qwen3-30B-A3B.
- `src/workload/generator.py`: Parameterized synthetic trace generator supporting Zipf skew and domain clustering.
- `calibration/stream_scaling_bench.cpp`: Native C++20 CXLMemSim stream scaling benchmark.
- `calibration/batch_activation_validation.cpp`: Native C++20 multi-stream batch activation validation harness.
- `results/`: Raw JSON experimental results for EXP-01 through EXP-05B across deterministic random seeds (42, 100, 2026).
- `analysis/`: Automated plotting, statistical validation, and report generation pipelines.